

Reviewer Pack — Noncommutative Geometry of Quantum Channels vs BP_ReadTwice ...

Entry 19e619289050 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 15:01:40 UTC

Noncommutative Geometry of Quantum Channels vs BP_ReadTwice Circuit Size

Entry ID: 19e619289050 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 15:01:40 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Quantum Channels) **Field B** (complexity object): Complexity Theory: BP_ReadTwice Complexity

Statement:

[‘For any quantum channel Φ , the minimal index of noncommutativity $\rho(\Phi)$ is upper-bounded by $O(\log \text{size}(\text{BP_}\Phi))$ and lower-bounded by $\Omega(n/\log n)$, where $\text{BP_}\Phi$ is the corresponding read-twice branching program.’, ‘ $\rho(\Phi) = \Theta(\log \text{size}(\text{BP_}\Phi))$ ’, ‘ $\rho(\Phi) \geq C \log n / \log \log n$ for some constant C .’]

Rationale (proposer’s reasoning):

[‘Quantum channels capture the essence of quantum computations, and their noncommutativity is a fundamental property that could potentially lead to non-trivial lower bounds on circuit complexity.’, ‘Noncommutative geometry offers tools for quantifying the non-commutativity of quantum systems, which might be applicable to branching programs representing these channels.’, ‘Previous work has shown that certain invariants from non-commutative geometry can relate to classical complexity theory. This conjecture aims to explore this relationship further with a specific focus on BP_ReadTwice circuit size.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b89be6b3071b53de

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all quantum channels Φ , the minimal index of noncommutativity $\rho(\Phi)$ falls within the bounds $O(\log \text{size}(\text{BP_}\Phi))$ and $\Omega(n/\log n)$, with a support fraction ≥ 0.8 across at least 30 seeds. The criterion is falsified if any seed results in $\rho(\Phi)$ outside these bounds or if the mean metric value deviates from the expected by more than 3 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    matrix = [row[:] for row in matrix]
    gaussian_elimination(matrix)
    rank = 0
    for i in range(m):
        if any(matrix[i]):
            rank += 1
    return rank

def commutator(A, B):
    return matrix_multiply(matrix_multiply(A, B), -matrix_multiply(B, A))

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    result = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(len(B)):
```

```

        result[i][j] += A[i][k] * B[k][j]
    return result

def identity_matrix(size):
    return [[1 if i == j else 0 for j in range(size)] for i in range(size)]

def random_quantum_channel(n):
    channel = []
    for _ in range(n):
        row = [random.random() for _ in range(n)]
        channel.append(row)
    return channel

def minimal_index_of_noncommutativity(channel):
    identity = identity_matrix(len(channel))
    rho = rank(commutator(identity, channel))
    return rho

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rho = 0
    instances_tested = 0

    for n in n_values:
        channel = random_quantum_channel(n)
        rho = minimal_index_of_noncommutativity(channel)
        total_rho += rho
        instances_tested += 1

    mean_rho = total_rho / len(n_values)
    conjecture_holds = all(5 <= rho <= 20 for rho in [minimal_index_of_noncommutativity(random_quantum_channel(n)) for n in n_values])

    return {
        "metric_name": "minimal_index_of_noncommutativity",
        "metric_value": mean_rho,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rho = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_rho) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_rho} std={std_dev} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rho} std={std_dev} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_020b2779.py", line 95, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_020b2779.py", line 74, in run_trial
    rho = minimal_index_of_noncommutativity(channel)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_020b2779.py", line 63, in minimal_index_of_noncommutativity
    rho = rank(commutator(identity, channel))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_020b2779.py", line 40, in commutator
    return matrix_multiply(matrix_multiply(A, B), -matrix_multiply(B, A))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12305
2	propose	ollama_remote	glm4:latest	0	0	9758
3	propose	ollama_remote	glm4:latest	0	0	10252
4	preregistration	ollama_remote	glm4:latest	0	0	6529

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	glm4:latest	0	0	4918
6	novelty	ollama_remote	glm4:latest	0	0	5100
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16548
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11955
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10807
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10997
11	judge	ollama_remote	glm4:latest	0	0	8212

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 107381 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/19e619289050.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/19e619289050.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/19e619289050.tar.gz* (if generated)